Szablon sprawozdania laboratoryjnego

****Przedmiot:**** Uczenie głębokie do rozpoznawania obiektów

****Zajęcia nr / temat:**** 3

****Autor / grupa:**** Mateusz Pluta / GL5.03

****Data:**** 28.10.2026r.

1. Cel ćwiczenia

Celem zadania było zbudowanie od podstaw potoku danych (data pipeline) do przetwarzania obrazów w PyTorchu. Wymagało to pobrania zbioru zdjęć, załadowania go przy pomocy klasy ImageFolder, przygotowania transformacji (osobnych dla danych treningowych i testowych) oraz przepuszczenia całości przez DataLoader. Dodatkowo należało wyciągnąć statystyki zbioru i pokazać na wykresach, jak działa augmentacja.

2. Metoda i implementacja

****Wykorzystane narzędzia, modele i zbiory danych****

PyTorch, torchvision, kagglehub, PIL, matplotlib

****Najważniejsze fragmenty kodu (lub link do repozytorium/notatnika)****

https://github.com/mattty1/Uczenie-glebokie-do-rozpoznawania-obiektow.git

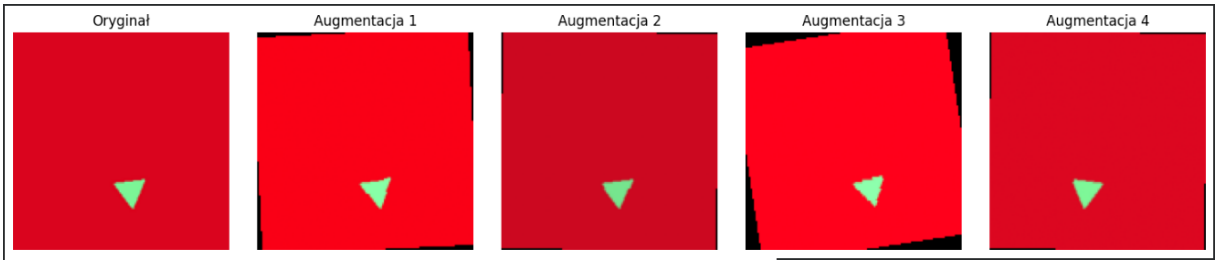
****Konfiguracja (architektura, hiperparametry, augmentacje)****

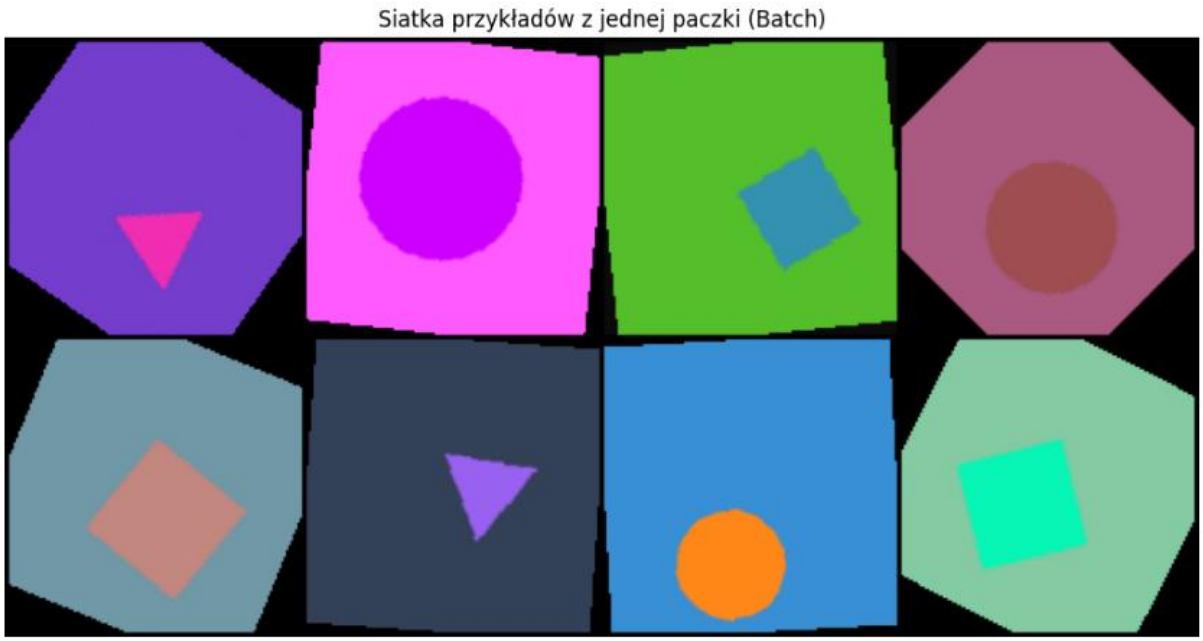
Wszystkie zdjęcia są skalowane do rozmiaru 128x128 pikseli. Zbiór ewaluacyjny (testowy) jest tylko skalowany i zamieniany na tensor. W zbiorze treningowym dodano augmentację: losowe obroty (do 45 stopni), odbicia lustrzane i zmiany jasności/kontrastu. Ma to uchronić przyszły model przed uczeniem się na pamięć i poprawić jego generalizację.

3. Wyniki

Skrypt poprawnie rysuje dwa wykresy. Pierwszy pokazuje, jak działają nałożone transformacje – widać, że w 4 wygenerowanych próbkach program losowo obraca figurę i zmienia jej jasność. Drugi wykres to siatka 8 losowych zdjęć pobrana prosto z batcha przez obiekt DataLoader.

Wynik w konsoli potwierdza, że PyTorch ustandaryzował wymiary do tensora [3, 128, 128] (3 kanały RGB, 128 wys., 128 szer.). Pokazuje też, że każda klasa dostała swój unikalny indeks całkowity.





4. Analiza i interpretacja

Wykorzystanie funkcji ImageFolder mocno ułatwiło pracę – nie trzeba było ręcznie pisać pętli do przypisywania etykiet, bo PyTorch zrobił to automatycznie na podstawie nazw folderów. Rozdzielenie transformacji okazało się kluczowe, dzięki temu zachowaliśmy pewność, że zbiór testowy nie zostanie przypadkowo zniekształcony podczas sprawdzania modelu.

Przy wizualizacji (funkcja imshow) wyszła ważna techniczna rzecz: biblioteka rysująca oczekuje formatu [Wysokość, Szerokość, Kanały], a ToTensor() w PyTorchu robi z tego [Kanały, Wysokość, Szerokość]. Żeby wyświetlić paczkę zdjęć z DataLoadera, trzeba było odwrócić ten proces za pomocą metody .permute(1, 2, 0)

5. Wnioski

Zbudowany potok działa sprawnie: automatycznie wczytuje dane, przydziela im etykiety, dorzuca augmentację na etapie treningu i grupuje obrazy w 8-elementowe paczki (batche). Tak przygotowany skrypt to w pełni funkcjonalna baza, do której można bezpośrednio dopiąć architekturę sieci neuronowej i rozpocząć uczenie.

Kryteria oceny (rubryka)

| Element | Waga |

|---|---|

| Poprawność implementacji | 40% |

| Wyniki i ich interpretacja | 30% |

| Czytelność kodu (PEP8, komentarze) | 20% |

| Wnioski | 10% |

Próg zaliczenia sprawozdań (O1): 90%. Oddanie: do tygodnia po zajęciach.